Neovim Cheatsheet SPC leader • Snacks Ivy • Flash • LSP + DAP • mini.ai • One Dark • Rust / Go / Python / TS / F# / Lua

ERGONOMIC STRATEGIES

SPC leader	thumb on Space; home row never moves
zz on jumps	cursor re-centers; eyes stay mid-screen
Flash s+2ch	jump anywhere visible; replaces f/t/;/,
unnamedplus	y/p = system clipboard; no "+" prefix
j/k=gj/gk	visual-line motion; natural for wrapped code
:w auto-fmt	conform.nvim runs on save; no manual step
800ms / SPC?	which-key hints; query prefix groups live
Snacks Ivy	picker at bottom; SPC sR resumes last
+/- visual	treesitter node expand / shrink in place

CORE MOTION

h j k l	left / down / up / right
w / b / e	word fwd / back / end
W / B / E	WORD fwd / back / end
ge / gE	back to prev word / WORD end
0 / ^ / $	line start / first non-blank / end
gg / G	file top / bottom
{ / }	prev / next blank line
%	jump matching bracket or delimiter
H / M / L	screen top / mid / bottom
C-u / C-d	scroll half-page up / down
C-f / C-b	scroll full page fwd / back
zz / zt / zb	cursor: center / top / bottom of screen

ON-LINE CHAR MOTION

f{c}	jump to next occurrence of char c
F{c}	jump to prev occurrence of char c
t{c}	jump before next char c
T{c}	jump before prev char c
; / ,	repeat last f/t fwd / reverse

Flash s covers multi-line hops; f/t stay useful within one line.

FLASH.NVIM n / v / o

s	Flash jump — 2 chars then pick label
S	Flash Treesitter — jump to syntax node
r	Remote flash (operator-pending only)
R	Treesitter search (op / visual)
C-s	Toggle Flash inside / ? search

CENTERED JUMP LIST all +zz

n / N	next / prev search result
* / #	word fwd / back (whole-word match)
g* / g#	word fwd / back (partial match)
C-o / C-i	older / newer jump position
" / "	last jump: line / precise position
'. / `.	last change: line / precise position
`[ / `]	prev change/yank: start / end
' /] '	prev change line: start / end
C-]	jump to tag definition
C-t	pop tag stack (back)
g; / g,	older / newer changelist position

BRACKET & SCOPE NAV

[(/ [{	prev unmatched (/ {
]) /]}	next unmatched) / }
[m /]m	prev / next method start
[M /]M	prev / next method end
gn / gN	select next / prev search match
cgn / dgn	change / delete next match (dot-repeatable)

MARKS

m{a-z}	set lowercase mark in current file
m{A-Z}	set global mark (across files)
`{a-z}	jump to mark — precise position
'{a-z}	jump to mark — start of line
SPC sm	marks picker (Snacks)

REGISTERS REFERENCE

""	unnamed: last delete / yank / change
"0	last explicit yank (y command)
"1-"9	last 9 deletes, newest = 1
"-	last small delete (<1 line)
".	last inserted text
":	last executed ex command
"/	last search pattern
"+	system clipboard (read and write)
"%	current filename
"=	expression register (prompt on use)

Access any register: "{r}y to yank, "{r}p to paste.

REGISTER TRICKS

"_d{m}	delete to black hole — no clobber
"0p	paste last yank (not last delete)
C-r 0	insert mode: paste last-yank inline
C-r {r}	insert mode: paste any register
:reg	list all register contents

Yank first, delete freely. "0p always pastes the original yank.

VISUAL & SELECTION

v / V / C-v	char / line / block visual
J / K	move selection down / up (visual)
< / >	indent left / right
gv	re-select last visual region
o	toggle to other end of selection
O	toggle to other corner (block visual)

VISUAL BLOCK C-v operations

I{t} Esc	insert {t} before every selected line
A{t} Esc	append {t} after every selected line
$ A{t} Esc	append past end of varying-length lines
c{t} Esc	replace block with {t} on all lines
r{c}	replace every char in block with c
~	toggle case of all chars in block

TREESITTER SELECTION v / op

+	expand to parent syntax node
-	shrink to child syntax node
gS	Flash Treesitter in visual (alt to S)

INSERT MODE keys while inserting

C-h	delete char before cursor (backspace)
C-w	delete word before cursor
C-u	delete to start of line
C-t / C-d	indent / de-indent current line
C-r {r}	paste from register r inline
C-o	one normal cmd, then back to insert
C-c	exit insert (no abbreviation trigger)
Esc / C-[	return to normal mode

In insert, C-x C-f = filename completion; C-x C-l = whole-line.

COMMAND-LINE MODE after : / ? /

C-f	open history in editable buffer
C-r C-w	paste word under cursor
C-r "	paste unnamed register
C-r /	paste last search pattern
C-b / C-e	cursor to line start / end
C-p / C-n	history prev / next
Tab	cycle completions (wildmode)
Esc	cancel, return to normal

q: / q/ open history as an editable normal-mode window.

NORMAL MODE QOL

.	repeat last change (most powerful)
@:	repeat last ex command
q: / q/	command / search history window
C-a / C-x	increment / decrement number
r{c}	replace char under cursor
R	enter replace mode (overwrite)
x / X	delete char at / before cursor
gi	go to last insert position and enter insert
gv	re-select last visual region
ga	show ASCII / Unicode of char
gf	open file name under cursor
gx	open URL or path externally

CASE & FORMAT OPERATORS

~	toggle case of char under cursor
g~{m}	toggle case by motion
gu{m}	lowercase by motion
gU{m}	uppercase by motion
gq{m}	reflow / wrap text to textwidth
J / gJ	join line below (with / no space)

UNDO / REDO undofile = true (persistent)

u / C-r	undo / redo
U	undo all changes on current line
g- / g+	undo / redo by time state
:earlier {t}	back by time (e.g. 5m 3h)
:later {t}	forward by time
SPC su	undo history picker (Snacks)

MACROS & RECORDING

q{a-z}	start recording into register
q{A-Z}	append recording to register
q	stop recording
@{a-z}	replay macro from register
@@	replay last-used macro
N@{a-z}	replay N times (count prefix)
:{a-z}	replay macro from cmd line

FOLDS

za / zA	toggle fold / toggle recursively
zc / zo	close / open fold
zC / zO	close / open all nested folds
zR / zM	open all / close all folds
zf{m}	create manual fold by motion
zd / zD	delete fold / delete all folds
[z /]z	move to fold start / fold end

SPELL CHECKING requires :set spell

]s / [s	next / prev misspelled word
z=	suggest spelling corrections
zg	add word to spellfile (good)
zw	mark word as wrong
zug / zuw	undo zg / undo zw
:set spell	enable spell checking

LSP — NAVIGATION

K	hover documentation
gd	goto definition (Snacks picker)
gD	goto declaration
gr	find references (Snacks picker)
gI	goto implementation
gy	goto type definition
SPC la	code action
SPC ln	rename symbol
SPC lr	LSP references (buffer list)
SPC lt	type definition
SPC ss	LSP symbols — buffer
SPC sS	LSP workspace symbols

DIAGNOSTICS

[d /]d	prev / next diagnostic
SPC xd	show line diagnostic (float)
SPC xt	toggle inline virtual text
SPC sd	diagnostics picker (all files)

COMPLETION nvim-cmp, insert mode

C-Space	trigger completion menu
C-n / C-p	next / prev completion item
CR	confirm selection
C-e	close / abort completion
C-b / C-f	scroll docs up / down
Tab / S-Tab	next / prev snippet placeholder

Ghost-text preview appears inline (Neovim 0.10+). Completion sources: LSP + LuaSnip.

OPERATORS + motion or text object

c	change	d	delete
y	yank	>	indent →
<	indent ←	=	auto-indent
gc	line comment	gb	block comment
ys	surround add		

Examples: `yaf` *yank outer fn* • `cit` *change in tag* • `dap` *delete paragraph*

TEXT OBJECTS mini.ai — prefix a / i

f	function (treesitter)
c	class (treesitter)
o	block / conditional / loop (TS)
t	HTML/XML tag
b	bracket pair ( ) [ ] { }
q	quote " ' `
w / W	word / WORD
p	paragraph
g	entire file (mini.ai bonus)

COMMENTS & SURROUND

gcc	toggle line comment
gbc	toggle block comment
gc{m}	comment by motion (e.g. gcip)
gco / gcO	comment line below / above
gcA	append comment at end of line
ys{m}{c}	add surround (e.g. ysiw")
yss{c}	surround entire current line
ds{c}	delete surrounding char
cs{o}{n}	change surround (e.g. cs"')
S{c}	surround visual selection

S may conflict with Flash Treesitter in visual — use `gS` *as fallback.*

EX RANGES & GLOBAL

:%	entire file (shorthand for 1,$)
:1,10	lines 1 to 10
:'<,'>	visual selection range
:.,.+5	current line to +5 lines
:g/pat/{cmd}	cmd on lines matching pat
:v/pat/{cmd}	cmd on lines NOT matching
:!{cmd}	run shell command
:{r}!{cmd}	filter range through cmd
:read !{cmd}	insert cmd output below

Examples: `:g/TODO/d` *delete all TODO lines* • `:%!jq .` *pretty-print JSON*

EX POWER COMMANDS

:sort	sort lines asc (u=unique, n=numeric)
:sort!	sort descending
:%y+	yank entire file to system clipboard
:put {r}	paste register as new lines below cursor
:norm {cmd}	run normal cmd on every line in range
:retab	replace tabs with spaces (per tabstop)
:saveas {f}	save buffer to new filename

`:,+3norm A;` *appends* `;` *to next 4 lines* • `:%y+` *then paste in browser.*

SEARCH & SUBSTITUTE

/pat	forward search (regex)
?pat	backward search (regex)
:%s/o/n/gc	replace all with confirm
:g/pat/cmd	global cmd on matching lines
:v/pat/cmd	global cmd on non-matching
C-r C-w	paste word under cursor (cmdline)
C-l	clear search highlights + redraw

REGEX SYNTAX / ? :s patterns

\v	very magic: ( ) | + work unescaped
\c / \C	force case-insensitive / case-sensitive
\< / \>	word boundary: start / end
\zs / \ze	set match start / end (trims result)
\%V	match only inside last visual selection
\n	newline in search; \r = newline in repl
\1..\9	back-reference to captured group

Prefix patterns with `\v` *to avoid extra escaping:* `/\v(\w+)\s+\1` *finds doubled words.*

FILES Snacks.nvim

SPC fe	file explorer (sidebar)
SPC ff	find files
SPC fr	recent files
SPC fs	smart find (files + recent + bufs)

BUFFERS

SPC bl	buffer list picker
SPC bp / bn	previous / next buffer
SPC bd	delete buffer
SPC w	quick save (:w)

SEARCH PICKERS Snacks Ivy — bottom

SPC sb	buffer lines (fuzzy)
SPC sg	grep (ripgrep)
SPC sw	grep word / visual sel. (n/v)
SPC sC	commands
SPC sh	help pages
SPC sj	jump list
SPC sk	keymaps
SPC sl	location list
SPC sm	marks
SPC sq	quickfix list
SPC sr	registers
SPC sR	resume last picker
SPC su	undo history
SPC ?	buffer-local keymaps

WINDOWS & SPLITS

C-w s / v	horizontal / vertical split
C-w h/j/k/l	navigate to adjacent window
C-w H/J/K/L	move window to edge
C-w r / R	rotate windows cw / ccw
C-w x	exchange with next window
C-w T	move window to new tab
C-w =	equalize window sizes
C-w q / C-w o	close window / close others

DIFF MODE :windo diffthis to start

]c / [c	next / prev diff hunk
do / dp	obtain diff from / put diff to other
:diffupdate	refresh diff highlighting
:windo diffthis	diff all visible windows
:diffoff	exit diff mode (current window)
:diffoff!	exit diff mode (all windows)

Open two files side-by-side: `:vs file2` *then* `:windo diffthis`. *Or* `nvim -d f1 f2` *from CLI.*

QUICKFIX & LOCLIST

]q / [q	next / prev quickfix item
]l / [l	next / prev location list item
SPC sq	quickfix picker (Snacks)
SPC sl	location list picker (Snacks)
:copen / :cclose	open / close quickfix window
:cdo {cmd}	run cmd on every quickfix entry
:cfdo {cmd}	run cmd on each quickfix file

`:cdo %s/old/new/g` *bulk-renames across all quickfix matches.*

TABS & TERMINAL

:tabnew	open new tab
gt / gT	next / prev tab
N gt	jump to tab N
:tabclose	close current tab
:term	open terminal buffer
C-\ C-n	terminal: exit to normal mode

DEBUG nvim-dap + dap-ui (auto open/close)

SPC dc / F5	start / continue
SPC do / F10	step over
SPC di / F11	step into
SPC du / F12	step out
SPC db	toggle breakpoint
SPC dB	conditional breakpoint (prompt)
SPC dr	open debug REPL
SPC dl	run last debug session

RUST rustaceanvim — rust files only

SPC rr	runnables
SPC rt	testables
SPC rd	debuggables (codelldb)
SPC re	expand macro inline
SPC rc	open Cargo.toml
SPC rp	goto parent module
SPC rj	join lines (smart)
K	hover + actions (richer override)
SPC la	code action (richer override)

Clippy on save. All cargo features + proc macros enabled. Inlay hints on.

LANG-SPECIFIC DEBUG

SPC dtm	Python: debug test method
SPC dtc	Python: debug test class
SPC dt	Go: debug nearest test (Delve)

EDITOR BEHAVIORS automatic

BufWritePre	trailing whitespace stripped on save
BufReadPost	cursor position restored on open
TextYankPost	yanked region highlighted 200 ms
VimResized	splits auto-resize with window
undofile	persistent undo history across sessions
exrc+secure	.nvimrc in project root auto-loaded
Inlay hints	auto-enabled for Rust + F# on attach
autowrite	buffer saved on window switch / :make

COMMON :SET OPTIONS

:set nu	line numbers on (nonu = off)
:set rnu	relative line numbers
:set wrap	wrap long lines (nowrap = off)
:set spell	spell checking (see SPELL section)
:set list	show tabs and trailing spaces
:set cc=80	vertical column guide at col 80
:set paste	paste mode — disables auto-indent

Prefix with `no` *to disable:* `:set nospell`. `:setlocal` *applies buffer-only.*

PLUGIN COMMANDS

:Mason	open Mason package manager UI
:LspInfo	show attached LSP servers
:LspRestart	restart LSP for current buffer
:ConformInfo	show formatter configuration
:TSUpdate	update all treesitter parsers
:checkhealth	Neovim + plugin health check

GITSIGNS :Gitsigns {cmd}

stage_hunk	stage diff hunk under cursor
reset_hunk	discard hunk to HEAD
preview_hunk	floating diff preview
blame_line	git blame for current line
diffthis	diff buffer vs index
toggle_signs	toggle gutter sign column

Signs active in gutter. No default keymaps — use `:Gitsigns {cmd}` *directly.*